



Robótica y Percepción Computacional

Introducción a Numpy

Luis Baumela
Departamento de Inteligencia Artificial
Universidad Politécnica de Madrid



POLITÉCNICA

Introducción

El entorno científico



SciPy (pronounced "Sigh Pie") is a Python-based ecosystem of open-source software for mathematics, science, and engineering. In particular, these are some of the core packages:



NumPy

Base N-dimensional
array package



SciPy library

Fundamental library for
scientific computing



Matplotlib

Comprehensive 2D
Plotting

IP[y]:
IPython

IPython

Enhanced Interactive
Console



Sympy

Symbolic mathematics



pandas

Data structures &
analysis

<https://www.scipy.org/about.html>

Numpy is the core library for scientific computing in Python.

It provides:

- a high-performance multidimensional array object,
- tools for working with these arrays.

A numpy array is a grid of values, all of the same type

- It is **indexed by a tuple** .
- The number of dimensions is the **rank** of the array;
- the **shape** of an array is a tuple of integers giving the size of the array along each dimension

We can initialize numpy arrays from nested Python lists, and access elements using square brackets.

```
import numpy as np

a = np.array([1, 2, 3]) # Create a rank 1 array
print type(a)          # Prints "<type 'numpy.ndarray'>"
print a.shape          # Prints "(3,)"
print a[0], a[1], a[2] # Prints "1 2 3"
a[0] = 5               # Change an element of the array
print a                # Prints "[5, 2, 3]"

b = np.array([[1,2,3],[4,5,6]]) # Create a rank 2 array
print b.shape              # Prints "(2, 3)"
print b[0, 0], b[0, 1], b[1, 0] # Prints "1 2 4"
```

```
(tf-gpu2) iago@iago-GL63-8RCS:~$ ipython
Python 3.7.4 (default, Aug 13 2019, 20:35:49)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.7.0 -- An enhanced Interactive Python. Type '?' for help.
```

```
In [1]: import numpy as np
```

```
In [2]: np.array([[1, 2], [3, 4]])
```

```
Out[2]:
array([[1, 2],
       [3, 4]])
```

```
In [3]: A = np.array([[1, 2], [3, 4]])
```

```
In [4]: A
```

```
Out[4]:
array([[1, 2],
       [3, 4]])
```

```
In [5]: A.shape
```

```
Out[5]: (2, 2)
```

```
In [6]: A.size
```

```
Out[6]: 4
```

```
In [7]: A.ndim
```

```
Out[7]: 2
```

```
In [8]: B = np.array([[1, 2], [3, 4], [5, 6]])
```

```
In [9]: B
```

```
Out[9]:
array([[1, 2],
       [3, 4],
       [5, 6]])
```

```
In [10]:
```

- ¿B.shape?
- ¿B.size?
- ¿B.ndim?

```
(tf-gpu2) iago@iago-GL63-8RCS:~$ ipython
Python 3.7.4 (default, Aug 13 2019, 20:35:49)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.7.0 -- An enhanced Interactive Python. Type '?' for help.
```

```
In [1]: import numpy as np
```

```
In [2]: np.array([[1, 2], [3, 4]])
```

```
Out[2]:
array([[1, 2],
       [3, 4]])
```

```
In [3]: A = np.array([[1, 2], [3, 4]])
```

```
In [4]: A
```

```
Out[4]:
array([[1, 2],
       [3, 4]])
```

```
In [5]: A.shape
```

```
Out[5]: (2, 2)
```

```
In [6]: A.size
```

```
Out[6]: 4
```

```
In [7]: A.ndim
```

```
Out[7]: 2
```

```
In [8]: B = np.array([[1, 2], [3, 4], [5, 6]])
```

```
In [9]: B
```

```
Out[9]:
array([[1, 2],
       [3, 4],
       [5, 6]])
```

```
In [10]:
```

- B.shape = (3, 2)
- B.size = 6
- B.ndim = 2

Array indexing: **Slicing**

```
import numpy as np

# Create the following rank 2 array with shape (3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

# Use slicing to pull out the subarray consisting of the first 2 rows
# and columns 1 and 2; b is the following array of shape (2, 2):
# [[2 3]
#  [6 7]]
b = a[:2, 1:3]

# A slice of an array is a view into the same data, so modifying it
# will modify the original array.
print a[0, 1]    # Prints "2"
b[0, 0] = 77     # b[0, 0] is the same piece of data as a[0, 1]
print a[0, 1]    # Prints "77"
```


Array indexing: **Slicing** with index arrays

- Index arrays must be of integer type.
- Returns an array with the same shape as the index array, but with the type and values of the array being indexed.

```
In [6]: a=np.array([[1,0],[2,3],[4,2]])  
print(a)  
print(a.shape)
```

```
[[1 0]  
 [2 3]  
 [4 2]]  
(3, 2)
```

```
In [7]: paleta = np.array([[0,0,0],[0,0,255],[255,0,0],[0,255,0],[0,0,0]],dtype=np.uint8)
```

```
In [8]: paleta[a].shape
```

```
Out[8]: (3, 2, 3)
```

```
In [9]: print(paleta[a])
```

```
[[[ 0  0 255]  
  [ 0  0  0]]  
  
 [[255  0  0]  
  [ 0 255  0]]  
  
 [[ 0  0  0]  
  [255  0  0]]]
```


Array indexing: **Slicing** with index arrays

```
import numpy as np

a = np.array([[1,2], [3, 4], [5, 6]])

# An example of integer array indexing.
# The returned array will have shape (3,) and
print a[[0, 1, 2], [0, 1, 0]] # Prints "[1 4 5]"

# The above example of integer array indexing is equivalent to this:
print np.array([a[0, 0], a[1, 1], a[2, 0]]) # Prints "[1 4 5]"

# When using integer array indexing, you can reuse the same
# element from the source array:
print a[[0, 0], [1, 1]] # Prints "[2 2]"

# Equivalent to the previous integer array indexing example
print np.array([a[0, 1], a[0, 1]]) # Prints "[2 2]"
```

Array indexing: **Boolean indexing** or **mask** arrays

- Boolean arrays must be of the same shape as the initial dimensions of the array being indexed.
- Returns 1-D array with all the elements in the indexed array corresponding to all the true elements in the boolean array.

Array indexing: **Boolean indexing** or **mask** arrays

```
import numpy as np

a = np.array([[1,2], [3, 4], [5, 6]])

bool_idx = (a > 2) # Find the elements of a that are bigger than 2;
                  # this returns a numpy array of Booleans of the same
                  # shape as a, where each slot of bool_idx tells
                  # whether that element of a is > 2.

print bool_idx      # Prints "[False False]
                   #           [ True  True]
                   #           [ True  True]]"

# We use boolean array indexing to construct a rank 1 array
# consisting of the elements of a corresponding to the True values
# of bool_idx
print a[bool_idx] # Prints "[3 4 5 6]"

# We can do all of the above in a single concise statement:
print a[a > 2]    # Prints "[3 4 5 6]"
```

Array data types

- Every numpy array is a grid of elements of the same type.
- Numpy provides a large set of numeric datatypes that you can use to construct arrays.

```
import numpy as np

x = np.array([1, 2]) # Let numpy choose the datatype
print x.dtype       # Prints "int64"

x = np.array([1.0, 2.0]) # Let numpy choose the datatype
print x.dtype          # Prints "float64"

x = np.array([1, 2], dtype=np.int64) # Force a particular datatype
print x.dtype                 # Prints "int64"
```

Array maths

- Basic mathematical functions operate elementwise on arrays, and are available both as operator overloads and as functions.

```
import numpy as np

x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

# Elementwise sum; both produce the array
# [[ 6.0  8.0]
#  [10.0 12.0]]
print x + y
print np.add(x, y)

# Elementwise difference; both produce the array
# [[-4.0 -4.0]
#  [-4.0 -4.0]]
print x - y
print np.subtract(x, y)

# Elementwise product; both produce the array
# [[ 5.0 12.0]
#  [21.0 32.0]]
print x * y
print np.multiply(x, y)
```

Array maths

- Algebraic array multiplications are performed with the `dot` function.

```
import numpy as np

x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11, 12])

# Inner product of vectors; both produce 219
print v.dot(w)
print np.dot(v, w)

# Matrix / vector product; both produce the rank 1 array
print x.dot(v)
print np.dot(x, v)

# Matrix / matrix product; both produce the rank 2 array
# [[19 22]
#  [43 50]]
print x.dot(y)
print np.dot(x, y)
```

Python 3:

```
In [24]:
In [24]: x = np.array([[1, 2], [3, 4]])
In [25]: y = np.array([[5, 6], [7, 8]])
In [26]: x.dot(y)
Out[26]:
array([[19, 22],
       [43, 50]])
In [27]: x @ y
Out[27]:
array([[19, 22],
       [43, 50]])
In [28]:
```

Broadcasting

- Allows numpy to work with arrays of different shapes when performing arithmetic operations.
- Provides a means of vectorizing array operations. It does this without making needless copies of data and usually leads to efficient algorithm implementations.

Approach:

1. NumPy compares their shapes element-wise. It starts with the trailing dimensions, and works its way forward.
Two dimensions are compatible when
 - they are equal, or
 - one of them is 1
2. The size of the resulting array is the maximum size along each dimension of the input arrays.

Broadcasting

```
import numpy as np

# We will add the vector v to each row of the matrix x,
# storing the result in the matrix y
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
y = x + v # Add v to each row of x using broadcasting
print y # Prints "[[ 2  2  4]
        #           [ 5  5  7]
        #           [ 8  8 10]
        #           [11 11 13]]"
```

Broadcasting

```
import numpy as np

# Compute outer product of vectors
v = np.array([1,2,3]) # v has shape (3,)
w = np.array([4,5])   # w has shape (2,)
x = np.array([[1,2,3], [4,5,6]])

# Add a vector to each column of a matrix
# x has shape (2, 3) and w has shape (2,).
# If we transpose x then it has shape (3, 2) and can be broadcast
# against w to yield a result of shape (3, 2); transposing this result
# yields the final result of shape (2, 3) which is the matrix x with
# the vector w added to each column. Gives the following matrix:
# [[ 5  6  7]
#  [ 9 10 11]]
print (x.T + w).T
```

Broadcasting

- Para reorganizar los ejes de un tensor es útil

`np.moveaxis(<tensor>, [source], [destination])`

```
>>> x = np.zeros((3, 4, 5))

>>> np.transpose(x).shape
(5, 4, 3)
>>> np.swapaxes(x, 0, -1).shape
(5, 4, 3)
>>> np.moveaxis(x, [0, 1], [-1, -2]).shape
(5, 4, 3)
```

- Para añadir nuevos ejes

`np.newaxis` o `None`

```
>>> newaxis is None
True
>>> x = np.arange(3)
>>> x
array([0, 1, 2])
>>> x[:, newaxis]
array([[0],
       [1],
       [2]])
```

```
>>> x[:, newaxis, newaxis]
array([[[0]],
       [[1]],
       [[2]])]
>>> x[:, newaxis] * x
array([[0, 0, 0],
       [0, 1, 2],
       [0, 2, 4]])
```

Tareas para el proyecto de la asignatura

Etiquetado de una imagen

Objetivo: Lee un vídeo y selecciona una imagen para etiquetar manualmente suelo, línea y marca.

Imagen capturada

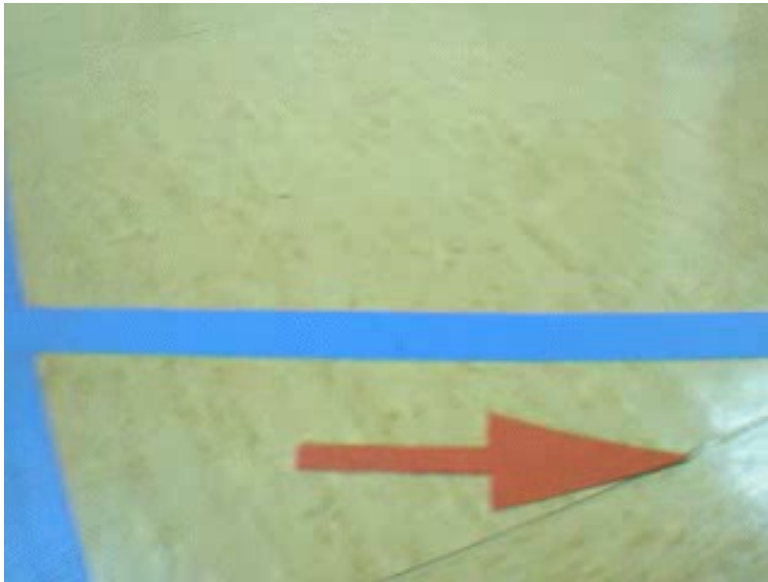
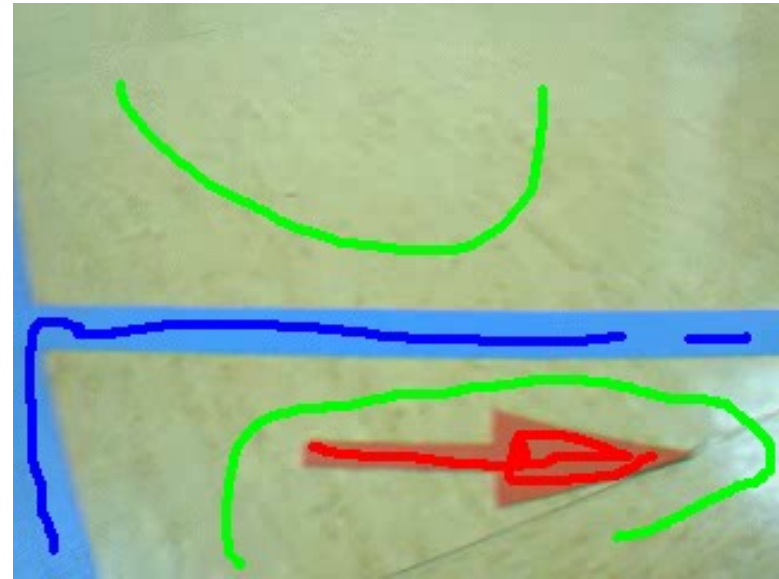


Imagen marcada



Tareas para el proyecto de la asignatura

Etiquetado de una imagen

Objetivo: Lee un vídeo y selecciona una imagen para etiquetar manualmente suelo, línea y marca. Ver archivos de código:

etiqueta_imagenes.ipynb
select_pixels.py

```
import sys
sys.path.append('./python_code')
```

```
import cv2
# Python 2.7
# from scipy.misc import imread, imsave
# Para Python > 3.5 o para Python < 3.5 ->
from imageio import imread, imsave
from matplotlib import pyplot as plt
import select_pixels as sel
```

```
# Transformo la imagen al formato Python
imNumpy=cv2.cvtColor(im,cv2.COLOR_BGR2RGB)

# Llamo a la funcion para etiquetar la imagen
markImg = sel.select_fg_bg(imNumpy)

plt.imshow(markImg)
plt.show()
```

```
# Guardo las imagenes
imsave('imgs/xxx.png',imNumpy)
imsave('imgsxxy.png',markImg)
```

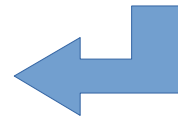
```
#Captura de imagenes
capture = cv2.VideoCapture('videos/xxxx.avi')

while (key != ...):
    # Leo una imagen del video
    ret,im = capture.read()

    if not ret:
        print('Se acabo el video.')
        cv2.destroyAllWindows()
        break

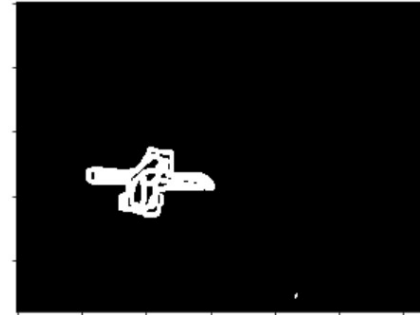
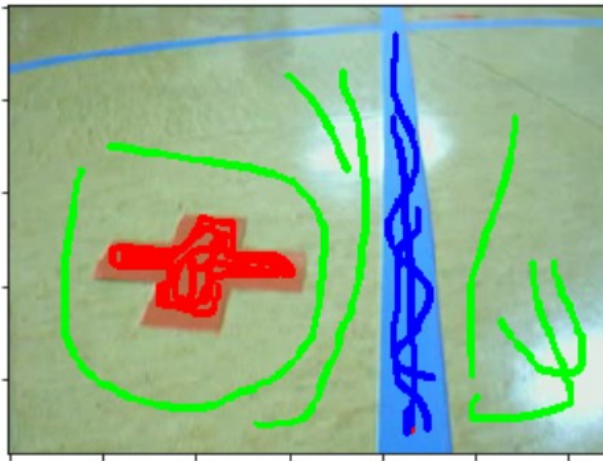
    # Muestro la imagen en una ventala OpenCV
    cv2.imshow("Captura", im)

    key = cv2.waitKey(35)
```



Objetivo: Muestra los píxeles marcados con un color.

- Busca en la imagen marcada los píxeles etiquetados en un color.



Tareas para el proyecto de la asignatura

Distribuciones de color

Objetivo: Aplica la normalización RGB a una imagen.

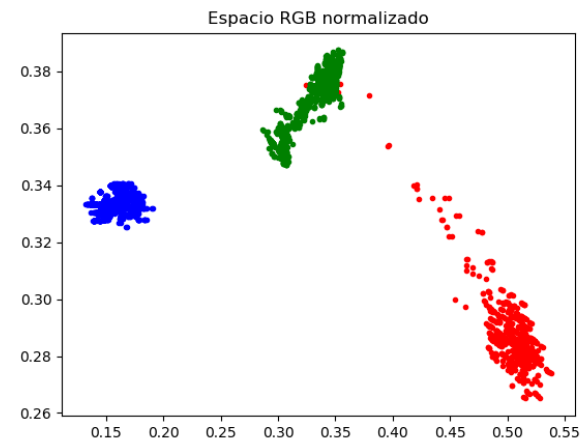
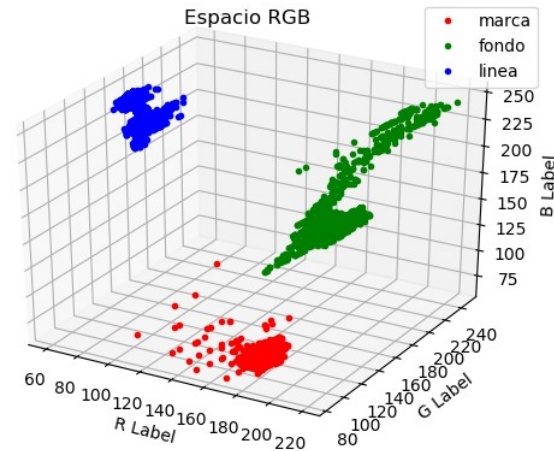
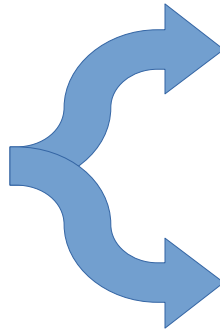
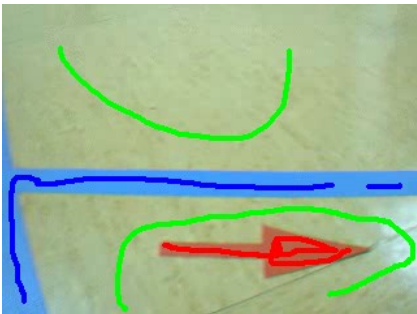
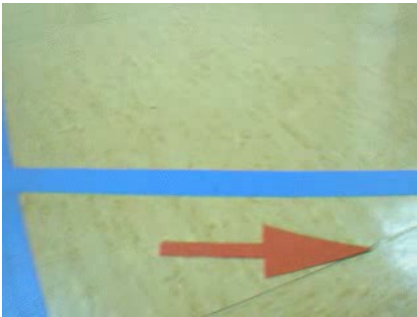
- Divide el valor RGB de cada píxel por la suma de las tres componentes.

Construye un programa que lea un vídeo y genere otro con los canales de color normalizados.

Tareas para el proyecto de la asignatura

Distribuciones de color

Objetivo: Muestra las distribuciones de color de marca, línea y suelo.



Tareas para el proyecto de la asignatura

Distribuciones de color

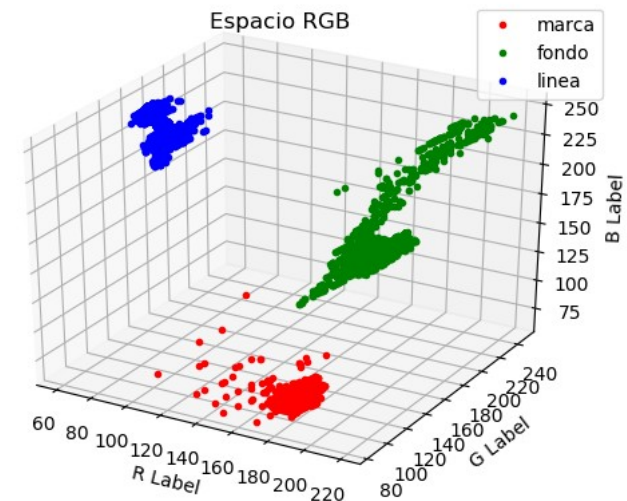
Objetivo: Muestra las distribuciones de color de marca, línea y suelo.

- Lee de la imagen original los valores rgb de los píxeles etiquetados en la imagen marcada.
- Pinta dichos valores en un espacio RGB

```
# Almaceno los valores RGB de 'imNp' que fueron marcados en 'markImg' en rojo
data_marca = # TODO datos marcados en rojo
data_fondo = # TODO datos marcados en verde
data_linea = # TODO datos marcados en azul
```

```
# Pinto los datos RGB
fig = plt.figure()
ax = fig.add_subplot(projection='3d')

ax.plot(data_marca[:,0],data_marca[:,1],data_marca[:,2], 'r.', label='marca')
ax.plot(data_fondo[:,0],data_fondo[:,1],data_fondo[:,2], 'g.', label='fondo')
ax.plot(data_linea[:,0],data_linea[:,1],data_linea[:,2], 'b.', label='linea')
ax.set_xlabel('R Label')
ax.set_ylabel('G Label')
ax.set_zlabel('B Label')
ax.legend()
plt.title('Espacio RGB')
```



Tareas para el proyecto de la asignatura

Distribuciones de color

Objetivo: Muestra las distribuciones de color **normalizadas** de marca, línea y suelo.

- Lee de la imagen normalizada los valores rgb de los píxeles etiquetados en la imagen marcada.
- Pinta dichos valores en un espacio RGB

```
# Almaceno los valores RGB de 'imrgbn' que fueron marcados en 'markImg'  
data_marca_n = # datos marcados en rojo  
data_fondo_n = # datos marcados en verde  
data_linea_n = # datos marcados en azul
```

```
plt.figure()  
plt.plot(data_marca_n[:,0],data_marca_n[:,1], 'r.', label='marca')  
plt.plot(data_fondo_n[:,0],data_fondo_n[:,1], 'g.', label='fondo')  
plt.plot(data_linea_n[:,0],data_linea_n[:,1], 'b.', label='linea')  
plt.title('Espacio RGB normalizado')
```

